Lab 3: SQL数据处理和分析实习

任务成员

毛川2300013218

关睿轩2300013238

芮思铭2300013094

Task 1: SQL数据预处理实习任务

1. 环境准备与数据库创建

首先导入必要的库并创建SQLite数据库连接。

In [1]:
```
import sqlite3
import pandas as pd
import numpy as np

conn = sqlite3.connect('user_cleaning.db')
cursor = conn.cursor()
```

2. 创建原始数据表

创建包含脏数据的用户原始数据表，模拟真实业务场景中的各种数据质量问题。

In [2]:
```
cursor.execute('DROP TABLE IF EXISTS user_raw_data')

cursor.execute('''
CREATE TABLE user_raw_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    raw_name TEXT,           -- 原始姓名（含空格、大小写混乱）
    raw_phone TEXT,          -- 原始手机号（含符号、空格、长度错误）
    raw_email TEXT,          -- 原始邮箱（格式不规范、冗余字符）
    raw_register_time TEXT,  -- 原始注册时间（格式混乱）
    raw_address TEXT,        -- 原始地址（含冗余字符、空格）
    raw_age TEXT,            -- 原始年龄（含文字、符号、空值）
    raw_remark TEXT          -- 备注（含特殊字符、空值）
)
''')
```

Out[2]:
```
<sqlite3.Cursor at 0x1e82b5975c0>
```

3. 插入脏测试数据

插入包含各种脏数据的测试数据，覆盖所有需要清洗的场景：

- 姓名：前后空格、中间空格、英文大小写混乱
- 手机号：含横杠、空格、字母、86前缀、长度不足
- 邮箱：缺少@、双@、冗余字符、大小写混乱
- 注册时间：多种日期格式混搭、异常日期
- 地址：含前后空格
- 年龄：含'岁'字、'未知'、空值
- 备注：含空格、NULL值、空字符串

In [3]:
```
# 插入脏测试数据
raw_data = [
    ('张三', '13800138000', 'zhangsan@163.com', '2024-01-15 09:30:00', '北京市朝
    (' 李四 ', '13912345678', 'lisi@gmail.com', '2024/02/20', '上海市浦东新区张江
    ('王 五', '137-8888-9999', 'wangwu@qq..com', '2024.03.10', '广州市天河区珠江
    ('Zhao Liu', ' 13677778888 ', 'zhaoliu123', '2024-04-05', '深圳市南山区科技园
    ('钱七', '8613566667777', 'qianqi@outlook.com', '2024/05/12 10:00', '杭州市西
    ('孙八', '1345555', 'sunba@126.com', '2024.06.18 14:20', '成都市高新区天府大
    ('周九', '133abc123456', 'zhoujiu@', '2024-07-22', '武汉市洪山区光谷广场', '
    ('吴十', '13211112222', 'WU_SHI@@qq.com', '2024/08/30', '重庆市渝中区解放碑'
    ('郑十一', '13100001111', 'zhengeshiyi123@', '2024.09.05', '西安市雁塔区高新
    ('王十二', '13099998888', 'wangshier@163.com', '2024年10月01日', '长沙市岳麓
    ('李十三', '12988887777', 'lisan@qq.com', '2024-13-01', '青岛市市南区香港中路
    ('张十四', '', 'zhangsi@126.com', '', '昆明市五华区南屏街', '未知', '')
]

cursor.executemany('''
INSERT INTO user_raw_data
(raw_name, raw_phone, raw_email, raw_register_time,
 raw_address, raw_age, raw_remark)
VALUES (?, ?, ?, ?, ?, ?, ?)
''', raw_data)

conn.commit()
```

4. 查看原始数据

查看插入的原始数据，了解脏数据的实际情况。

In [4]:
```
print('原始数据')

raw_df = pd.read_sql_query(
    'SELECT * FROM user_raw_data',
    conn
)
raw_df
```

原始数据

Out[4]:

	id	raw_name	raw_phone	raw_email	raw_register_time	raw_ad
0	1	张三	13800138000	zhangsan@163.com	2024-01-15 09:30:00	北京市朝 建国路
1	2	李四	13912345678	lisi@gmail.com	2024/02/20	上海市浦 区张江高
2	3	王 五	137-8888-9999	wangwu@qq..com	2024.03.10	广州市天 珠江
3	4	Zhao Liu	13677778888	zhaoliu123	2024-04-05	深圳市南 科
4	5	钱七	8613566667777	qianqi@outlook.com	2024/05/12 10:00	杭州市西 西溪
5	6	孙八	1345555	sunba@126.com	2024.06.18 14:20	成都市高 天府
6	7	周九	133abc123456	zhoujiu@	2024-07-22	武汉市洪 光谷
7	8	吴十	13211112222	WU_SHI@@qq.com	2024/08/30	重庆市渝 解
8	9	郑十一	13100001111	zhengeshiyi123@	2024.09.05	西安市雁 高
9	10	王十二	13099998888	wangshier@163.com	2024年10月01日	长沙市岳 大
10	11	李十三	12988887777	lisan@qq.com	2024-13-01	青岛市市 香港
11	12	张十四		zhangsi@126.com		昆明市五 商

5. 数据清洗与标准化

清洗规则说明

1. **姓名标准化**：去除姓名中的前后空格和中间多余空格
2. **手机号清洗**：去除横杠、空格、字母等冗余字符，只保留11位数字
3. **邮箱标准化**：转换为小写，校验格式（必须包含@和.）
4. **日期标准化**：将各种日期格式统一转换为YYYY-MM-DD格式
5. **地址标准化**：去除地址中的前后空格
6. **年龄标准化**：提取纯数字年龄，非数字标记为"未知年龄"
7. **备注标准化**：空值、NULL、空格统一替换为"无备注"

新增清洗任务

8. **姓名长度校验**：检查姓名长度是否合法（≥2个字符）
9. **手机号运营商识别**：根据手机号前三位识别运营商
10. **邮箱服务商提取**：识别QQ邮箱、网易邮箱、Gmail等

11. **用户年龄分类**：将用户分为青年、中年等年龄段
12. **地址省份提取**：从地址中提取省份信息

In [6]:
```
# 创建标准化数据表
cursor.execute('DROP TABLE IF EXISTS user_standard_data')

# SQLite的REGEXP函数需要手动启用，这里使用LIKE和GLOB替代
# 注意：SQLite中默认没有REGEXP_REPLACE函数，这里使用嵌套REPLACE处理

cursor.execute('''
CREATE TABLE user_standard_data AS
SELECT
    id,

    -- 任务1：姓名标准化（去除空格）
    TRIM(REPLACE(raw_name, ' ', '')) AS user_name,

    -- 任务2：手机号清洗
    CASE
        WHEN LENGTH(
            REPLACE(
                REPLACE(
                    REPLACE(
                        REPLACE(raw_phone, '-', ''),
                    ' ', ''),
                'abc', ''),
            '86', '')
        ) = 11
        THEN REPLACE(
                REPLACE(
                    REPLACE(
                        REPLACE(raw_phone, '-', ''),
                    ' ', ''),
                'abc', ''),
            '86', '')
        ELSE '无效手机号'
    END AS user_phone,

    -- 任务3：邮箱标准化
    CASE
        WHEN raw_email LIKE '%@%.%'
             AND raw_email NOT LIKE '%@@%'
             AND raw_email NOT LIKE '%..%'
             AND raw_email NOT LIKE '%@%@%'
        THEN LOWER(raw_email)
        ELSE '无效邮箱'
    END AS user_email,

    -- 任务4：日期标准化
    CASE
        WHEN raw_register_time != '' AND raw_register_time IS NOT NULL
        THEN SUBSTR(
                REPLACE(
                    REPLACE(
                        REPLACE(
                            REPLACE(raw_register_time, '年', '-'),
                        '月', '-'),
                    '日', ''),
                '/', '-'),
```

```
            1, 10)
        ELSE '无效日期'
    END AS register_date,

    -- 任务5：地址标准化
    TRIM(raw_address) AS user_address,

    -- 任务6：年龄标准化
    CASE
        WHEN raw_age IN ('未知', '', NULL) OR raw_age IS NULL
        THEN '未知年龄'
        ELSE REPLACE(raw_age, '岁', '')
    END AS user_age,

    -- 任务7：备注标准化
    CASE
        WHEN TRIM(raw_remark) = ''
             OR TRIM(raw_remark) = 'NULL'
             OR raw_remark IS NULL
        THEN '无备注'
        ELSE TRIM(raw_remark)
    END AS user_remark,

    -- ================================================
    -- 新增任务1：姓名长度校验
    -- ================================================
    CASE
        WHEN LENGTH(TRIM(REPLACE(raw_name, ' ', ''))) >= 2
        THEN '姓名合法'
        ELSE '姓名异常'
    END AS name_status,

    -- ================================================
    -- 新增任务2：手机号运营商识别
    -- ================================================
    CASE
        WHEN raw_phone LIKE '138%' OR raw_phone LIKE '139%'
             OR raw_phone LIKE '188%' OR raw_phone LIKE '187%'
             OR raw_phone LIKE '135%' OR raw_phone LIKE '136%'
             OR raw_phone LIKE '137%' OR raw_phone LIKE '150%'
             OR raw_phone LIKE '151%' OR raw_phone LIKE '152%'
        THEN '中国移动'
        WHEN raw_phone LIKE '130%' OR raw_phone LIKE '131%'
             OR raw_phone LIKE '132%' OR raw_phone LIKE '155%'
             OR raw_phone LIKE '156%' OR raw_phone LIKE '185%'
             OR raw_phone LIKE '186%'
        THEN '中国联通'
        WHEN raw_phone LIKE '133%' OR raw_phone LIKE '153%'
             OR raw_phone LIKE '180%' OR raw_phone LIKE '181%'
             OR raw_phone LIKE '189%'
        THEN '中国电信'
        ELSE '未知运营商'
    END AS operator_type,

    -- ================================================
    -- 新增任务3：邮箱服务商提取
    -- ================================================
    CASE
        WHEN raw_email LIKE '%qq.com%' THEN 'QQ邮箱'
        WHEN raw_email LIKE '%163.com%' THEN '网易邮箱'
```

```
        WHEN raw_email LIKE '%126.com%' THEN '网易邮箱'
        WHEN raw_email LIKE '%gmail.com%' THEN 'Gmail邮箱'
        WHEN raw_email LIKE '%outlook.com%' THEN 'Outlook邮箱'
        ELSE '其他邮箱'
    END AS email_provider,

    -- ================================================
    -- 新增任务4：用户年龄分类
    -- ================================================
    CASE
        WHEN REPLACE(raw_age, '岁', '') GLOB '[0-9]*'
             AND CAST(REPLACE(raw_age, '岁', '') AS INTEGER) < 30
        THEN '青年'
        WHEN REPLACE(raw_age, '岁', '') GLOB '[0-9]*'
             AND CAST(REPLACE(raw_age, '岁', '') AS INTEGER) BETWEEN 30 AND 40
        THEN '中年'
        WHEN REPLACE(raw_age, '岁', '') GLOB '[0-9]*'
             AND CAST(REPLACE(raw_age, '岁', '') AS INTEGER) > 40
        THEN '中老年'
        ELSE '未知'
    END AS age_group,

    -- ================================================
    -- 新增任务5：地址省份提取
    -- ================================================
    CASE
        WHEN raw_address LIKE '北京市%' THEN '北京'
        WHEN raw_address LIKE '上海市%' THEN '上海'
        WHEN raw_address LIKE '广州市%' OR raw_address LIKE '深圳市%'
             OR raw_address LIKE '广东省%'
        THEN '广东'
        WHEN raw_address LIKE '杭州市%' OR raw_address LIKE '浙江省%'
        THEN '浙江'
        WHEN raw_address LIKE '成都市%' OR raw_address LIKE '四川省%'
        THEN '四川'
        WHEN raw_address LIKE '武汉市%' OR raw_address LIKE '湖北省%'
        THEN '湖北'
        WHEN raw_address LIKE '重庆市%' THEN '重庆'
        WHEN raw_address LIKE '西安市%' OR raw_address LIKE '陕西省%'
        THEN '陕西'
        WHEN raw_address LIKE '长沙市%' OR raw_address LIKE '湖南省%'
        THEN '湖南'
        WHEN raw_address LIKE '青岛市%' OR raw_address LIKE '山东省%'
        THEN '山东'
        WHEN raw_address LIKE '昆明市%' OR raw_address LIKE '云南省%'
        THEN '云南'
        ELSE '其他地区'
    END AS province

FROM user_raw_data
''')

conn.commit()
```

标准化数据表创建成功！

6. 查看清洗后的数据

查看经过清洗和标准化处理后的数据，对比原始数据可以看到明显的改善。

In [7]:
```
print('\n')
print('清洗后的标准化数据')

clean_df = pd.read_sql_query(
    'SELECT * FROM user_standard_data',
    conn
)

clean_df
```

```
============================================================
清洗后的标准化数据
============================================================
```

Out[7]:

	id	user_name	user_phone	user_email	register_date	user_address
0	1	张三	13800138000	zhangsan@163.com	2024-01-15	北京市朝阳区建国路88号
1	2	李四	13912345678	lisi@gmail.com	2024-02-20	上海市浦东新区张江高科技园区
2	3	王五	13788889999	无效邮箱	2024.03.10	广州市天河区珠江新城
3	4	ZhaoLiu	13677778888	无效邮箱	2024-04-05	深圳市南山区科技园
4	5	钱七	13566667777	qianqi@outlook.com	2024-05-12	杭州市西湖区西溪湿地
5	6	孙八	无效手机号	sunba@126.com	2024.06.18	成都市高新区天府大道
6	7	周九	无效手机号	无效邮箱	2024-07-22	武汉市洪山区光谷广场
7	8	吴十	13211112222	无效邮箱	2024-08-30	重庆市渝中区解放碑
8	9	郑十一	13100001111	无效邮箱	2024.09.05	西安市雁塔区高新区
9	10	王十二	13099998888	wangshier@163.com	2024-10-01	长沙市岳麓区大学城
10	11	李十三	12988887777	lisan@qq.com	2024-13-01	青岛市市南区香港中路
11	12	张十四	无效手机号	zhangsi@126.com	无效日期	昆明市五华区南屏街

7. 数据质量评估

对清洗前后的数据进行质量对比评估，包括：

- 缺失值统计
- 无效数据统计
- 数据完整性分析

In [8]:
```
print('\n')
print('数据质量评估')

print('\n【原始数据缺失值统计】')
missing_raw = raw_df.isnull().sum()
print(missing_raw)

print('\n【清洗后缺失值统计】')
missing_clean = clean_df.isnull().sum()
print(missing_clean)

invalid_phone = clean_df[clean_df['user_phone'] == '无效手机号'].shape[0]
print(f'\n无效手机号数量: {invalid_phone}')

invalid_email = clean_df[clean_df['user_email'] == '无效邮箱'].shape[0]
print(f'无效邮箱数量: {invalid_email}')

invalid_date = clean_df[clean_df['register_date'] == '无效日期'].shape[0]
print(f'无效日期数量: {invalid_date}')

unknown_age = clean_df[clean_df['user_age'] == '未知年龄'].shape[0]
print(f'未知年龄数量: {unknown_age}')

invalid_name = clean_df[clean_df['name_status'] == '姓名异常'].shape[0]
print(f'姓名异常数量: {invalid_name}')

print('\n【运营商分布】')
operator_dist = clean_df['operator_type'].value_counts()
print(operator_dist)

print('\n【年龄分组统计】')
age_group_dist = clean_df['age_group'].value_counts()
print(age_group_dist)

print('\n【省份分布】')
province_dist = clean_df['province'].value_counts()
print(province_dist)
```

```
============================================================
数据质量评估
============================================================

【原始数据缺失值统计】
id                  0
raw_name            0
raw_phone           0
raw_email           0
raw_register_time   0
raw_address         0
raw_age             0
raw_remark          0
dtype: int64

【清洗后缺失值统计】
id                0
user_name         0
user_phone        0
user_email        0
register_date     0
user_address      0
user_age          0
user_remark       0
name_status       0
operator_type     0
email_provider    0
age_group         0
province          0
dtype: int64

无效手机号数量: 3
无效邮箱数量: 5
无效日期数量: 1
未知年龄数量: 2
姓名异常数量: 0

【运营商分布】
operator_type
未知运营商    5
中国移动      3
中国联通      3
中国电信      1
Name: count, dtype: int64

【年龄分组统计】
age_group
青年    5
中年    5
未知    2
Name: count, dtype: int64

【省份分布】
province
广东    2
北京    1
上海    1
浙江    1
四川    1
```

```
湖北     1
重庆     1
陕西     1
湖南     1
山东     1
云南     1
Name: count, dtype: int64
```

8. 数据导出

将清洗前后的数据导出为CSV文件，便于后续分析和使用。

In [9]:
```
raw_df.to_csv('raw_user_data.csv', index=False, encoding='utf-8-sig')
clean_df.to_csv('clean_user_data.csv', index=False, encoding='utf-8-sig')

print('1. raw_user_data.csv - 原始数据')
print('2. clean_user_data.csv - 清洗后数据')
```

```
数据已导出：
1. raw_user_data.csv - 原始数据
2. clean_user_data.csv - 清洗后数据
```

9. 清洗效果分析

清洗前后对比

字段	清洗前问题	清洗后结果
姓名	包含空格、大小写混乱	去除所有空格，统一格式
手机号	包含横杠、字母、86前缀	仅保留11位数字，无效标记
邮箱	大小写混乱、格式错误	小写统一，格式校验
日期	多种格式混搭	统一为YYYY-MM-DD
年龄	包含"岁"字、"未知"	提取纯数字或标记未知
备注	空值、NULL	统一替换为"无备注"

新增分析维度

通过本次清洗，我们还新增了以下分析维度：

1. **姓名合法性校验**：识别异常姓名
2. **运营商识别**：分析用户手机号所属运营商
3. **邮箱服务商**：了解用户偏好的邮箱平台
4. **年龄分层**：对用户进行年龄分组
5. **地域分布**：提取用户所在省份

10. 关闭数据库连接

完成所有操作后，关闭数据库连接释放资源。

In [10]:
```
conn.close()
```

```
数据库连接已关闭
============================================================
数据预处理实习任务完成!
============================================================
```

In [ ]:

Task2: MovieLens + SQLite Analysis Assignment

This notebook performs the following tasks:

1. Build a SQLite database and import `movies.csv` and `ratings.csv`
2. Create indexes for analysis queries
3. Run 5 required SQL analysis tasks and display results
4. Export each query result to CSV
5. Validate outputs with assertions

In [1]:
```
import sqlite3
import csv
import math
from pathlib import Path
import pandas as pd
from IPython.display import display, HTML

pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 160)

base_dir = Path.cwd()
movies_candidates = sorted(base_dir.rglob('movies.csv'))
ratings_candidates = sorted(base_dir.rglob('ratings.csv'))

if not movies_candidates or not ratings_candidates:
    raise FileNotFoundError('movies.csv or ratings.csv not found. Please check y

movies_path = movies_candidates[0]
ratings_path = ratings_candidates[0]

if movies_path.parent != ratings_path.parent:
    raise RuntimeError(f'movies.csv and ratings.csv are not in the same folder:

data_dir = movies_path.parent
db_path = base_dir / 'movielens.db'
results_dir = base_dir / 'query_results'
results_dir.mkdir(parents=True, exist_ok=True)

print('base_dir =', base_dir)
print('data_dir =', data_dir)
print('movies_path exists =', movies_path.exists())
print('ratings_path exists =', ratings_path.exists())
print('db_path =', db_path)
print('results_dir =', results_dir)
```

```
base_dir = c:\Users\19412\Desktop\3-2\database\proj3
data_dir = c:\Users\19412\Desktop\3-2\database\proj3\movielen数据集
movies_path exists = True
ratings_path exists = True
db_path = c:\Users\19412\Desktop\3-2\database\proj3\movielens.db
results_dir = c:\Users\19412\Desktop\3-2\database\proj3\query_results
```

In [2]:

```
conn = sqlite3.connect(db_path)
conn.create_function('SQRT', 1, lambda x: None if x is None else math.sqrt(x))
cur = conn.cursor()

cur.executescript("""
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS ratings;
DROP TABLE IF EXISTS movie_genres;
DROP TABLE IF EXISTS movies;

CREATE TABLE movies (
    movieId INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    genres TEXT NOT NULL
);

CREATE TABLE ratings (
    userId INTEGER NOT NULL,
    movieId INTEGER NOT NULL,
    rating REAL NOT NULL,
    timestamp INTEGER,
    FOREIGN KEY (movieId) REFERENCES movies(movieId)
);

CREATE TABLE movie_genres (
    movieId INTEGER NOT NULL,
    genre TEXT NOT NULL,
    FOREIGN KEY (movieId) REFERENCES movies(movieId)
);
""")

movies_rows = []
movie_genre_rows = []
filtered_no_genre_tokens = 0
no_genre_tokens = {'no genre', 'no genres listed', '(no genres listed)'}

with movies_path.open('r', encoding='utf-8', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
        movie_id = int(row['movieId'])
        title = row['title']
        genres = row['genres']
        movies_rows.append((movie_id, title, genres))
        for g in genres.split('|'):
            g = g.strip()
            g_norm = g.lower()
            if not g or g_norm in no_genre_tokens:
                filtered_no_genre_tokens += 1
                continue
            movie_genre_rows.append((movie_id, g))

cur.executemany('INSERT INTO movies (movieId, title, genres) VALUES (?, ?, ?)',
cur.executemany('INSERT INTO movie_genres (movieId, genre) VALUES (?, ?)', movie

ratings_rows = []
with ratings_path.open('r', encoding='utf-8', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
```

```
        ratings_rows.append((
            int(row['userId']),
            int(row['movieId']),
            float(row['rating']),
            int(row['timestamp'])
        ))

cur.executemany('INSERT INTO ratings (userId, movieId, rating, timestamp) VALUES

cur.executescript("""
CREATE INDEX idx_ratings_movieId ON ratings(movieId);
CREATE INDEX idx_ratings_userId ON ratings(userId);
CREATE INDEX idx_movie_genres_movieId ON movie_genres(movieId);
CREATE INDEX idx_movie_genres_genre ON movie_genres(genre);
""")

conn.commit()

movie_cnt = cur.execute('SELECT COUNT(*) FROM movies').fetchone()[0]
rating_cnt = cur.execute('SELECT COUNT(*) FROM ratings').fetchone()[0]
movie_genre_cnt = cur.execute('SELECT COUNT(*) FROM movie_genres').fetchone()[0]

print('movies =', movie_cnt)
print('ratings =', rating_cnt)
print('movie_genres =', movie_genre_cnt)
print('filtered no-genre tokens =', filtered_no_genre_tokens)
```

```
movies = 10329
ratings = 105339
movie_genres = 23107
filtered no-genre tokens = 7
```

In [3]:

```
# Data import validation
movie_cnt = pd.read_sql_query('SELECT COUNT(*) AS c FROM movies', conn)['c'].ilc
rating_cnt = pd.read_sql_query('SELECT COUNT(*) AS c FROM ratings', conn)['c'].i
movie_genre_cnt = pd.read_sql_query('SELECT COUNT(*) AS c FROM movie_genres', cc
empty_genre_cnt = pd.read_sql_query(
    "SELECT COUNT(*) AS c FROM movie_genres WHERE genre IS NULL OR TRIM(genre) =
    conn
)['c'].iloc[0]
placeholder_genre_cnt = pd.read_sql_query(
    """
    SELECT COUNT(*) AS c
    FROM movie_genres
    WHERE LOWER(TRIM(genre)) IN ('no genre', 'no genres listed', '(no genres lis
    """,
    conn
)['c'].iloc[0]

assert movie_cnt == 10329, f'movies row count mismatch: {movie_cnt}'
assert rating_cnt == 105339, f'ratings row count mismatch: {rating_cnt}'
assert movie_genre_cnt > movie_cnt, 'movie_genres row count should be greater th
assert empty_genre_cnt == 0, f'Found empty genres: {empty_genre_cnt}'
assert placeholder_genre_cnt == 0, f'Found placeholder genres that should be exc

print('Data import validation passed')
```

```
Data import validation passed
```

In [4]:

```
def _safe_name(name: str) -> str:
    return ''.join(ch if ch.isalnum() or ch in ('-', '_') else '_' for ch in nam
```

```
def run_sql(
    sql: str,
    query_name: str,
    preview_rows: int = 20,
    show_all: bool | None = None,
    save_csv: bool = True
) -> pd.DataFrame:
    df = pd.read_sql_query(sql, conn)

    if save_csv:
        file_name = _safe_name(query_name) + '.csv'
        out_path = results_dir / file_name
        df.to_csv(out_path, index=False, encoding='utf-8-sig')
        print(f'CSV saved: {out_path}')

    if show_all is None:
        show_all = len(df) <= 2000

    if show_all:
        html = df.to_html(index=False)
        display(HTML(f"<div style='max-height:480px; overflow:auto; border:1px s
        print('Display mode: full table in scrollable view')
    else:
        display(df.head(preview_rows))
        print(f'Display mode: preview first {preview_rows} rows')

    print(f'rows = {len(df)}')
    return df
```

1) Top 10 movies by average rating (minimum 10 ratings)

In [5]:
```
q1_sql = """
SELECT
    m.movieId,
    m.title,
    COUNT(*) AS rating_cnt,
    ROUND(AVG(r.rating), 4) AS avg_rating
FROM ratings r
JOIN movies m ON m.movieId = r.movieId
GROUP BY m.movieId, m.title
HAVING COUNT(*) >= 10
ORDER BY avg_rating DESC, rating_cnt DESC, m.title ASC
LIMIT 10;
"""

df_q1 = run_sql(q1_sql, query_name='q1_top10_movies_by_avg_rating', preview_rows
assert len(df_q1) == 10, f'Q1 result should have 10 rows, got {len(df_q1)}'
```

```
CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q1_top10_movie
s_by_avg_rating.csv
```

movieId	title	rating_cnt	avg_rating
1178	Paths of Glory (1957)	19	4.5000
1927	All Quiet on the Western Front (1930)	13	4.5000
1730	Kundun (1997)	10	4.5000
7099	Nausicaä of the Valley of the Wind (Kaze no tani no Naushika) (1984)	22	4.4773
1248	Touch of Evil (1958)	21	4.4762
3429	Creature Comforts (1989)	13	4.4615
1172	Cinema Paradiso (Nuovo cinema Paradiso) (1989)	37	4.4595
318	Shawshank Redemption, The (1994)	308	4.4545
66934	Dr. Horrible's Sing-Along Blog (2008)	23	4.4348
1949	Man for All Seasons, A (1966)	11	4.4091

Display mode: full table in scrollable view
rows = 10

2) Top 10 movies by average rating within each genre (minimum 10 ratings)

In [6]:
```
q2_sql = """
WITH movie_genre_stats AS (
    SELECT
        mg.genre,
        m.movieId,
        m.title,
        COUNT(*) AS rating_cnt,
        AVG(r.rating) AS avg_rating
    FROM ratings r
    JOIN movies m ON m.movieId = r.movieId
    JOIN movie_genres mg ON mg.movieId = m.movieId
    GROUP BY mg.genre, m.movieId, m.title
    HAVING COUNT(*) >= 10
),
ranked AS (
    SELECT
        genre,
        movieId,
        title,
        rating_cnt,
        ROUND(avg_rating, 4) AS avg_rating,
        ROW_NUMBER() OVER (
            PARTITION BY genre
            ORDER BY avg_rating DESC, rating_cnt DESC, title ASC
        ) AS genre_rank
    FROM movie_genre_stats
)
SELECT *
FROM ranked
WHERE genre_rank <= 10
ORDER BY genre ASC, genre_rank ASC;
```

```
"""

df_q2 = run_sql(q2_sql, query_name='q2_top10_movies_per_genre', preview_rows=40,

assert (df_q2['genre_rank'] <= 10).all(), 'Q2 has genre_rank > 10'
for genre, sub in df_q2.groupby('genre'):
    ranks = sorted(sub['genre_rank'].tolist())
    assert ranks == list(range(1, len(ranks) + 1)), f'Q2 genre {genre} rank is n
```

```
CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q2_top10_movie
s_per_genre.csv
```

genre	movieId	title	rating_cnt	avg_rating	genre_rank
Action	1927	All Quiet on the Western Front (1930)	13	4.5000	1
Action	3000	Princess Mononoke (Mononoke-hime) (1997)	52	4.3846	2
Action	3265	Hard-Boiled (Lat sau san taam) (1992)	13	4.3077	3
Action	908	North by Northwest (1959)	73	4.2740	4
Action	1224	Henry V (1989)	22	4.2727	5
Action	5782	Professional, The (Le professionnel) (1981)	22	4.2727	6
Action	2571	Matrix, The (1999)	261	4.2644	7
Action	1209	Once Upon a Time in the West (C'era una volta il	23	4.2609	8

```
Display mode: full table in scrollable view
rows = 190
```

3) Top 5 genres per user by overall evaluation (rating-count aware weighted score)

Overall score formula used in SQL

`overall_score = ((rating_cnt * avg_rating) + (3 * user_avg_rating)) / (rating_cnt + 3)`

Term definitions

- `avg_rating` : this user's average rating within the current genre
- `rating_cnt` : how many ratings this user has in the current genre
- `user_avg_rating` : this user's baseline average rating across all rated movies
- `3` : smoothing strength (prior weight)

Low-count genres are pulled toward the user baseline, while high-count genres depend more on genre-specific ratings.

In [7]:

```
q3_sql = """
WITH user_base AS (
    SELECT
        userId,
        AVG(rating) AS user_avg_rating
    FROM ratings
    GROUP BY userId
),
user_genre_stats AS (
    SELECT
        r.userId,
        mg.genre,
        COUNT(*) AS rating_cnt,
        AVG(r.rating) AS avg_rating
    FROM ratings r
    JOIN movie_genres mg ON mg.movieId = r.movieId
    GROUP BY r.userId, mg.genre
    HAVING COUNT(*) >= 2
),
scored AS (
    SELECT
        ugs.userId,
        ugs.genre,
        ugs.rating_cnt,
        ROUND(ugs.avg_rating, 4) AS avg_rating,
        ROUND(
            ((ugs.rating_cnt * ugs.avg_rating) + (3 * ub.user_avg_rating))
            / (ugs.rating_cnt + 3),
            4
        ) AS overall_score
    FROM user_genre_stats ugs
    JOIN user_base ub ON ub.userId = ugs.userId
),
ranked AS (
    SELECT
        userId,
        genre,
        rating_cnt,
        avg_rating,
        overall_score,
        ROW_NUMBER() OVER (
            PARTITION BY userId
            ORDER BY overall_score DESC, rating_cnt DESC, avg_rating DESC, genre
        ) AS rank_in_user
    FROM scored
)
SELECT *
FROM ranked
WHERE rank_in_user <= 5
ORDER BY userId ASC, rank_in_user ASC;
"""

df_q3 = run_sql(q3_sql, query_name='q3_top5_genres_per_user_weighted_overall', p
assert (df_q3['rank_in_user'] <= 5).all(), 'Q3 has rank_in_user > 5'
```

```
CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q3_top5_genres
_per_user_weighted_overall.csv
```

	userId	genre	rating_cnt	avg_rating	overall_score	rank_in_user
0	1	Crime	31	4.2097	4.1584	1
1	1	War	10	4.2000	4.0681	2
2	1	Thriller	43	3.8721	3.8562	3
3	1	Drama	45	3.8444	3.8309	4
4	1	Action	46	3.8261	3.8140	5
5	2	Drama	11	4.3636	4.2635	1
6	2	Animation	2	4.5000	4.1379	2
7	2	Children	3	4.3333	4.1149	3
8	2	Crime	3	4.3333	4.1149	4
9	2	Fantasy	4	4.2500	4.0985	5
10	3	Mystery	4	4.2500	4.0548	1
11	3	Crime	12	4.0000	3.9589	2
12	3	Drama	36	3.9722	3.9586	3
13	3	Western	3	4.0000	3.8973	4
14	3	Horror	2	4.0000	3.8767	5
15	4	War	16	4.5625	4.4992	1
16	4	Animation	4	4.7500	4.4977	2
17	4	Mystery	10	4.4000	4.3449	3
18	4	Drama	76	4.3421	4.3352	4
19	4	Children	6	4.3333	4.2760	5
20	5	IMAX	9	4.2778	4.0043	1
21	5	Animation	21	4.0952	3.9813	2
22	5	Musical	11	4.0909	3.8965	3
23	5	Children	21	3.9048	3.8146	4
24	5	Fantasy	16	3.8438	3.7396	5
25	6	Romance	15	4.3667	4.3315	1
26	6	Crime	11	4.3636	4.3191	2
27	6	Sci-Fi	12	4.3333	4.2978	3
28	6	Musical	2	4.5000	4.2934	4
29	6	Drama	31	4.2581	4.2490	5

```
Display mode: preview first 30 rows
rows = 3340
```

4) Top 5 genres per user by watch count

In [8]:
```
q4_sql = """
WITH user_genre_watch AS (
    SELECT
        r.userId,
        mg.genre,
        COUNT(DISTINCT r.movieId) AS watch_cnt,
        AVG(r.rating) AS avg_rating
    FROM ratings r
    JOIN movie_genres mg ON mg.movieId = r.movieId
    GROUP BY r.userId, mg.genre
),
ranked AS (
    SELECT
        userId,
        genre,
        watch_cnt,
        ROUND(avg_rating, 4) AS avg_rating,
        ROW_NUMBER() OVER (
            PARTITION BY userId
            ORDER BY watch_cnt DESC, avg_rating DESC, genre ASC
        ) AS rank_in_user
    FROM user_genre_watch
)
SELECT *
FROM ranked
WHERE rank_in_user <= 5
ORDER BY userId ASC, rank_in_user ASC;
"""

df_q4 = run_sql(q4_sql, query_name='q4_top5_genres_per_user_by_watch_count', pre
assert (df_q4['rank_in_user'] <= 5).all(), 'Q4 has rank_in_user > 5'
```

```
CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q4_top5_genres
_per_user_by_watch_count.csv
```

	userId	genre	watch_cnt	avg_rating	rank_in_user
0	1	Action	46	3.8261	1
1	1	Drama	45	3.8444	2
2	1	Thriller	43	3.8721	3
3	1	Crime	31	4.2097	4
4	1	Adventure	31	3.6935	5
5	2	Thriller	12	3.9167	1
6	2	Drama	11	4.3636	2
7	2	Comedy	11	3.5455	3
8	2	Adventure	10	4.0000	4
9	2	Action	9	3.8889	5
10	3	Drama	36	3.9722	1
11	3	Comedy	35	3.7429	2
12	3	Romance	22	3.6364	3
13	3	Thriller	21	3.7143	4
14	3	Action	13	3.2308	5
15	4	Drama	76	4.3421	1
16	4	Comedy	46	4.0217	2
17	4	Romance	37	4.1622	3
18	4	Crime	18	3.9444	4
19	4	Thriller	18	3.7222	5
20	5	Comedy	45	3.4444	1
21	5	Adventure	22	3.5455	2
22	5	Animation	21	4.0952	3
23	5	Children	21	3.9048	4
24	5	Romance	21	3.7381	5
25	6	Drama	31	4.2581	1
26	6	Comedy	28	4.2143	2
27	6	Romance	15	4.3667	3
28	6	Adventure	14	4.2143	4
29	6	Thriller	13	4.0769	5

```
Display mode: preview first 30 rows
rows = 3340
```

5) Similar-interest user pairs

- Shared watch count per genre >= 3
- Number of qualified shared genres >= 2
- Stddev of rating differences on shared movies <= 0.3

In [9]:
```
q5_sql = """
WITH common_movies AS (
    SELECT
        r1.userId AS user_a,
        r2.userId AS user_b,
        r1.movieId AS movieId,
        ABS(r1.rating - r2.rating) AS diff
    FROM ratings r1
    JOIN ratings r2
      ON r1.movieId = r2.movieId
     AND r1.userId < r2.userId
),
pair_genre_overlap AS (
    SELECT
        cm.user_a,
        cm.user_b,
        mg.genre,
        COUNT(*) AS overlap_cnt
    FROM common_movies cm
    JOIN movie_genres mg ON mg.movieId = cm.movieId
    GROUP BY cm.user_a, cm.user_b, mg.genre
    HAVING COUNT(*) >= 3
),
pair_genre_diff AS (
    SELECT
        cm.user_a,
        cm.user_b,
        mg.genre,
        SQRT(MAX(AVG(cm.diff * cm.diff) - AVG(cm.diff) * AVG(cm.diff), 0)) AS ra
    FROM common_movies cm
    JOIN movie_genres mg ON mg.movieId = cm.movieId
    GROUP BY cm.user_a, cm.user_b, mg.genre
),
qualified_pair_genres AS (
    SELECT
        o.user_a,
        o.user_b,
        o.genre,
        o.overlap_cnt,
        d.rating_diff_stddev
    FROM pair_genre_overlap o
    JOIN pair_genre_diff d
      ON o.user_a = d.user_a
     AND o.user_b = d.user_b
     AND o.genre = d.genre
    WHERE d.rating_diff_stddev <= 0.3
),
pair_summary AS (
    SELECT
        user_a,
        user_b,
        COUNT(*) AS qualified_genre_count,
        SUM(overlap_cnt) AS total_overlap_in_qualified_genres,
        ROUND(AVG(rating_diff_stddev), 4) AS avg_rating_diff_stddev,
        MIN(overlap_cnt) AS min_overlap_cnt,
```

```
        MAX(rating_diff_stddev) AS max_rating_diff_stddev,
        GROUP_CONCAT(genre, ', ') AS genre_list
    FROM qualified_pair_genres
    GROUP BY user_a, user_b
    HAVING COUNT(*) >= 2
)
SELECT *
FROM pair_summary
ORDER BY qualified_genre_count DESC,
         total_overlap_in_qualified_genres DESC,
         avg_rating_diff_stddev ASC,
         user_a ASC,
         user_b ASC;
"""

df_q5 = run_sql(q5_sql, query_name='q5_similar_interest_user_pairs', preview_row

if len(df_q5) > 0:
    assert (df_q5['user_a'] < df_q5['user_b']).all(), 'Q5 has mirrored duplicate
    assert (df_q5['qualified_genre_count'] >= 2).all(), 'Q5 has qualified_genre_
    assert (df_q5['min_overlap_cnt'] >= 3).all(), 'Q5 has genres with overlap_cn
    assert (df_q5['max_rating_diff_stddev'] <= 0.3 + 1e-12).all(), 'Q5 has ratin

print('Q5 threshold validation passed')
```

```
CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q5_similar_int
erest_user_pairs.csv
```

	user_a	user_b	qualified_genre_count	total_overlap_in_qualified_genres	avg_rati
0	71	419	11	50	
1	87	513	10	66	
2	30	419	10	64	
3	423	554	10	50	
4	153	296	10	49	
5	306	531	10	46	
6	255	384	9	60	
7	57	163	9	57	
8	490	569	9	56	
9	234	554	9	52	

	user_a	user_b	qualified_genre_count	total_overlap_in_qualified_genres	avg_ratir
10	106	306	9	51	
11	166	306	9	49	
12	18	282	9	43	
13	196	335	9	40	
14	119	667	9	37	
15	389	604	9	36	
16	361	423	9	34	
17	57	419	9	33	
18	106	389	9	32	
19	1	83	8	73	

	user_a	user_b	qualified_genre_count	total_overlap_in_qualified_genres	avg_ratir
20	405	419	8	62	
21	57	628	8	56	
22	484	667	8	54	
23	83	642	8	46	
24	57	401	8	45	
25	296	314	8	45	
26	45	586	8	45	
27	238	365	8	43	
28	306	599	8	42	

	user_a	user_b	qualified_genre_count	total_overlap_in_qualified_genres	avg_rati
29	57	607	8	40	

```
Display mode: preview first 30 rows
rows = 21982
Q5 threshold validation passed
```

In [10]:
```
# Optional: close the connection (skip this if you still want to query)
# conn.close()
```

Task 4: SQL-Based Machine Learning on the World Happiness Dataset

This notebook implements the required SQL-based machine learning tasks on the World Happiness dataset.

1. Multivariate linear regression (`Score` as target)
2. Multi-level decision tree classification (`Score` into `high/middle/low`)
3. K-means clustering (`k=3`) with min-max normalization

All model computations are expressed with SQL queries; Python is used only to orchestrate execution and display results.

In [1]:
```
import sqlite3
import math
from pathlib import Path
import pandas as pd
from IPython.display import display

pd.set_option('display.max_columns', 50)
pd.set_option('display.width', 180)


def find_world_happiness_dir(start: Path) -> Path:
    candidates = [start] + list(start.parents)
    for root in candidates:
        p = root / 'world_happiness'
        if p.exists() and p.is_dir():
            return p
    raise FileNotFoundError('Could not locate world_happiness directory from cur


current_dir = Path.cwd()
world_happiness_dir = find_world_happiness_dir(current_dir)
out_db_path = current_dir / 'world_happiness_task4.db'

print(f'Current directory: {current_dir}')
print(f'Dataset directory: {world_happiness_dir}')
print(f'SQLite database: {out_db_path}')
```

```
Current directory: D:\Courses\sixth_semester\Database\lab\lab3\task4
Dataset directory: D:\Courses\sixth_semester\Database\lab\lab3\task4\world_happin
ess
SQLite database: D:\Courses\sixth_semester\Database\lab\lab3\task4\world_happines
s_task4.db
```

In [2]:
```
# 1) Load and harmonize 2015-2019 CSV files into a unified schema.
schema_columns = [
    'Overall_rank',
    'Country',
    'Score',
    'GDP_per_capita',
    'Social_support',
    'Healthy_life_expectancy',
```

```
    'Freedom_to_make_life_choices',
    'Generosity',
    'Perceptions_of_corruption',
    'Year'
]

column_maps = {
    '2015': {
        'Happiness Rank': 'Overall_rank',
        'Country': 'Country',
        'Happiness Score': 'Score',
        'Economy (GDP per Capita)': 'GDP_per_capita',
        'Family': 'Social_support',
        'Health (Life Expectancy)': 'Healthy_life_expectancy',
        'Freedom': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
        'Trust (Government Corruption)': 'Perceptions_of_corruption',
    },
    '2016': {
        'Happiness Rank': 'Overall_rank',
        'Country': 'Country',
        'Happiness Score': 'Score',
        'Economy (GDP per Capita)': 'GDP_per_capita',
        'Family': 'Social_support',
        'Health (Life Expectancy)': 'Healthy_life_expectancy',
        'Freedom': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
        'Trust (Government Corruption)': 'Perceptions_of_corruption',
    },
    '2017': {
        'Happiness.Rank': 'Overall_rank',
        'Country': 'Country',
        'Happiness.Score': 'Score',
        'Economy..GDP.per.Capita.': 'GDP_per_capita',
        'Family': 'Social_support',
        'Health..Life.Expectancy.': 'Healthy_life_expectancy',
        'Freedom': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
        'Trust..Government.Corruption.': 'Perceptions_of_corruption',
    },
    '2018': {
        'Overall rank': 'Overall_rank',
        'Country or region': 'Country',
        'Score': 'Score',
        'GDP per capita': 'GDP_per_capita',
        'Social support': 'Social_support',
        'Healthy life expectancy': 'Healthy_life_expectancy',
        'Freedom to make life choices': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
        'Perceptions of corruption': 'Perceptions_of_corruption',
    },
    '2019': {
        'Overall rank': 'Overall_rank',
        'Country or region': 'Country',
        'Score': 'Score',
        'GDP per capita': 'GDP_per_capita',
        'Social support': 'Social_support',
        'Healthy life expectancy': 'Healthy_life_expectancy',
        'Freedom to make life choices': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
```

```
        'Perceptions of corruption': 'Perceptions_of_corruption',
    }
}

frames = []
for csv_path in sorted(world_happiness_dir.glob('*.csv')):
    year = csv_path.stem
    if year not in column_maps:
        continue
    df = pd.read_csv(csv_path)
    rename_map = column_maps[year]
    selected = df[list(rename_map.keys())].rename(columns=rename_map).copy()
    selected['Year'] = int(year)
    frames.append(selected)

happiness_df = pd.concat(frames, ignore_index=True)

for col in schema_columns:
    if col not in happiness_df.columns:
        happiness_df[col] = None

numeric_cols = [c for c in schema_columns if c not in ('Country',)]
for c in numeric_cols:
    happiness_df[c] = pd.to_numeric(happiness_df[c], errors='coerce')

happiness_df = happiness_df[schema_columns].dropna(subset=[
    'Overall_rank', 'Country', 'Score', 'GDP_per_capita', 'Social_support',
    'Healthy_life_expectancy', 'Freedom_to_make_life_choices', 'Generosity',
    'Perceptions_of_corruption'
]).reset_index(drop=True)

print('Unified dataset shape:', happiness_df.shape)
display(happiness_df.head())
```

```
Unified dataset shape: (781, 10)
```

	Overall_rank	Country	Score	GDP_per_capita	Social_support	Healthy_life_exp
0	1	Switzerland	7.587	1.39651	1.34951	
1	2	Iceland	7.561	1.30232	1.40223	
2	3	Denmark	7.527	1.32548	1.36058	
3	4	Norway	7.522	1.45900	1.33095	
4	5	Canada	7.427	1.32629	1.32261	

In [3]:
```
# 2) Build SQLite table: happyness
conn = sqlite3.connect(out_db_path)
conn.create_function('SQRT', 1, lambda x: None if x is None else math.sqrt(x))
conn.create_function('POWER', 2, lambda x, y: None if x is None or y is None els
conn.create_function('LOG', 1, lambda x: None if x is None or float(x) <= 0 else
cur = conn.cursor()

cur.executescript('''
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS happyness;
CREATE TABLE happyness (
```

```
    Overall_rank INTEGER,
    Country TEXT,
    Score REAL,
    GDP_per_capita REAL,
    Social_support REAL,
    Healthy_life_expectancy REAL,
    Freedom_to_make_life_choices REAL,
    Generosity REAL,
    Perceptions_of_corruption REAL,
    Year INTEGER
);
''')

happiness_df.to_sql('happyness', conn, if_exists='append', index=False)

cur.executescript('''
CREATE INDEX IF NOT EXISTS idx_happyness_country ON happyness(Country);
CREATE INDEX IF NOT EXISTS idx_happyness_score ON happyness(Score);
''')

conn.commit()

profile_df = pd.read_sql_query('''
SELECT
    COUNT(*) AS n_rows,
    COUNT(DISTINCT Country) AS n_countries,
    MIN(Score) AS min_score,
    MAX(Score) AS max_score,
    AVG(Score) AS avg_score
FROM happyness;
''', conn)
display(profile_df)

sample_df = pd.read_sql_query('SELECT * FROM happyness ORDER BY Year, Overall_ra
display(sample_df)
```

	n_rows	n_countries	min_score	max_score	avg_score
0	781	170	2.693	7.769	5.377232

	Overall_rank	Country	Score	GDP_per_capita	Social_support	Healthy_life_exp
0	1	Switzerland	7.587	1.39651	1.34951	
1	2	Iceland	7.561	1.30232	1.40223	
2	3	Denmark	7.527	1.32548	1.36058	
3	4	Norway	7.522	1.45900	1.33095	
4	5	Canada	7.427	1.32629	1.32261	
5	6	Finland	7.406	1.29025	1.31826	
6	7	Netherlands	7.378	1.32944	1.28017	
7	8	Sweden	7.364	1.33171	1.28907	
8	9	New Zealand	7.286	1.25018	1.31967	
9	10	Australia	7.284	1.33358	1.30923	

A) Multivariate Linear Regression (SQL)

Target variable: `Score`

Features:

- `GDP_per_capita`
- `Social_support`
- `Healthy_life_expectancy`
- `Freedom_to_make_life_choices`
- `Perceptions_of_corruption`

Approach:

1. Min-max normalize features with SQL
2. Train weights with gradient descent where each gradient is computed by SQL
3. Evaluate regression losses in SQL

In [4]:
```
# Prepare normalized training table with SQL.
cur.executescript('''
DROP TABLE IF EXISTS regression_data;
CREATE TABLE regression_data AS
WITH stats AS (
    SELECT
        MIN(GDP_per_capita) AS min_gdp,
        MAX(GDP_per_capita) AS max_gdp,
        MIN(Social_support) AS min_social,
        MAX(Social_support) AS max_social,
        MIN(Healthy_life_expectancy) AS min_health,
        MAX(Healthy_life_expectancy) AS max_health,
        MIN(Freedom_to_make_life_choices) AS min_freedom,
        MAX(Freedom_to_make_life_choices) AS max_freedom,
        MIN(Perceptions_of_corruption) AS min_corr,
```

```
        MAX(Perceptions_of_corruption) AS max_corr
    FROM happyness
)
SELECT
    rowid AS sample_id,
    Score AS y,
    (GDP_per_capita - min_gdp) / NULLIF(max_gdp - min_gdp, 0) AS x1,
    (Social_support - min_social) / NULLIF(max_social - min_social, 0) AS x2,
    (Healthy_life_expectancy - min_health) / NULLIF(max_health - min_health, 0)
    (Freedom_to_make_life_choices - min_freedom) / NULLIF(max_freedom - min_free
    (Perceptions_of_corruption - min_corr) / NULLIF(max_corr - min_corr, 0) AS x
FROM happyness, stats;

DROP TABLE IF EXISTS lr_weights;
CREATE TABLE lr_weights (
    b0 REAL,
    b1 REAL,
    b2 REAL,
    b3 REAL,
    b4 REAL,
    b5 REAL
);
INSERT INTO lr_weights VALUES (0.0, 0.0, 0.0, 0.0, 0.0, 0.0);

DROP TABLE IF EXISTS lr_training_log;
CREATE TABLE lr_training_log (
    iter INTEGER,
    mse REAL
);
''')
conn.commit()

learning_rate = 0.08
iterations = 500

for i in range(iterations):
    grads = pd.read_sql_query('''
        WITH w AS (
            SELECT b0, b1, b2, b3, b4, b5 FROM lr_weights LIMIT 1
        )
        SELECT
            AVG(-2.0 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5))) AS g0
            AVG(-2.0 * x1 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
            AVG(-2.0 * x2 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
            AVG(-2.0 * x3 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
            AVG(-2.0 * x4 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
            AVG(-2.0 * x5 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
        FROM regression_data, w;
    ''', conn).iloc[0]

    cur.execute('''
        UPDATE lr_weights
        SET
            b0 = b0 - ?,
            b1 = b1 - ?,
            b2 = b2 - ?,
            b3 = b3 - ?,
            b4 = b4 - ?,
            b5 = b5 - ?;
    ''', tuple(learning_rate * float(grads[f'g{k}']) for k in range(6)))
```

```
    if i % 20 == 0 or i == iterations - 1:
        mse_value = pd.read_sql_query('''
            WITH w AS (SELECT * FROM lr_weights LIMIT 1)
            SELECT AVG(POWER(y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5), 2
            FROM regression_data, w;
        ''', conn)['mse'].iloc[0]
        cur.execute('INSERT INTO lr_training_log(iter, mse) VALUES (?, ?);', (i,

conn.commit()

weights_df = pd.read_sql_query('SELECT * FROM lr_weights;', conn)
loss_curve_df = pd.read_sql_query('SELECT * FROM lr_training_log ORDER BY iter;'

display(weights_df)
display(loss_curve_df.tail(10))
```

	b0	b1	b2	b3	b4	b5
0	2.225761	1.78704	1.165615	1.352721	1.136308	0.619361

	iter	mse
16	320	0.307263
17	340	0.307012
18	360	0.306786
19	380	0.306582
20	400	0.306397
21	420	0.306229
22	440	0.306075
23	460	0.305935
24	480	0.305807
25	499	0.305695

In [5]:
```
# Create prediction table and compute regression loss functions in SQL.
cur.executescript('''
DROP VIEW IF EXISTS lr_predictions;
CREATE VIEW lr_predictions AS
WITH w AS (SELECT * FROM lr_weights LIMIT 1)
SELECT
    d.sample_id,
    d.y AS y_true,
    (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5) AS y_pred
FROM regression_data d, w;
''')
conn.commit()

regression_losses = pd.read_sql_query('''
SELECT
    AVG(POWER(y_true - y_pred, 2)) AS mse,
    SQRT(AVG(POWER(y_true - y_pred, 2))) AS rmse,
    AVG(ABS(y_true - y_pred)) AS mae,
    AVG(ABS((y_true - y_pred) / NULLIF(y_true, 0.0))) * 100.0 AS mape,
```

```
    AVG(
        CASE
            WHEN ABS(y_true - y_pred) <= 1.0 THEN 0.5 * POWER(y_true - y_pred, 2
            ELSE ABS(y_true - y_pred) - 0.5
        END
    ) AS huber_loss
FROM lr_predictions;
''', conn)

display(regression_losses)

df_pred_preview = pd.read_sql_query('''
SELECT * FROM lr_predictions
ORDER BY ABS(y_true - y_pred) DESC
LIMIT 10;
''', conn)
display(df_pred_preview)
```

	mse	rmse	mae	mape	huber_loss
0	0.305695	0.552897	0.430378	8.521703	0.147942

	sample_id	y_true	y_pred
0	773	3.488	5.496157
1	615	3.590	5.501450
2	777	3.334	5.133028
3	620	3.408	5.148856
4	457	3.766	5.472434
5	234	5.440	3.841046
6	466	3.471	5.059566
7	154	3.465	5.043154
8	156	3.006	4.510196
9	778	3.231	4.718967

B) Multi-Level Decision Tree Classification (SQL + orchestration)

Task setup:

- Convert `Score` into three classes (`high` , `middle` , `low`)
- Use SQL to search split candidates and minimize weighted Gini impurity
- Grow a multi-level tree with anti-overfitting constraints:
  - `max_depth = 3`
  - `min_samples_split`
  - `min_samples_leaf`

In [6]:
```
# 1) Build classification dataset with three classes by score tertiles.
cur.executescript('''
```

```
DROP TABLE IF EXISTS dt_dataset;
CREATE TABLE dt_dataset AS
WITH ranked AS (
    SELECT
        rowid AS sample_id,
        Country,
        Score,
        GDP_per_capita,
        Social_support,
        Healthy_life_expectancy,
        Freedom_to_make_life_choices,
        Generosity,
        Perceptions_of_corruption,
        NTILE(3) OVER (ORDER BY Score DESC, Country ASC) AS tertile_bucket
    FROM happyness
)
SELECT
    sample_id,
    Country,
    Score,
    GDP_per_capita,
    Social_support,
    Healthy_life_expectancy,
    Freedom_to_make_life_choices,
    Generosity,
    Perceptions_of_corruption,
    CASE
        WHEN tertile_bucket = 1 THEN 'high'
        WHEN tertile_bucket = 2 THEN 'middle'
        ELSE 'low'
    END AS score_class
FROM ranked;

DROP TABLE IF EXISTS dt_long;
CREATE TABLE dt_long AS
SELECT sample_id, score_class, 'GDP_per_capita' AS feature, GDP_per_capita AS fe
UNION ALL
SELECT sample_id, score_class, 'Social_support' AS feature, Social_support AS fe
UNION ALL
SELECT sample_id, score_class, 'Healthy_life_expectancy' AS feature, Healthy_lif
UNION ALL
SELECT sample_id, score_class, 'Freedom_to_make_life_choices' AS feature, Freedo
UNION ALL
SELECT sample_id, score_class, 'Generosity' AS feature, Generosity AS feature_va
UNION ALL
SELECT sample_id, score_class, 'Perceptions_of_corruption' AS feature, Perceptio
''')
conn.commit()

class_dist = pd.read_sql_query('''
SELECT score_class, COUNT(*) AS n
FROM dt_dataset
GROUP BY score_class
ORDER BY n DESC;
''', conn)

display(class_dist)
```

	score_class	n
0	high	261
1	middle	260
2	low	260

In [7]:
```
# 2) Train a multi-level decision tree with regularization constraints.
max_depth = 3
min_samples_split = 30
min_samples_leaf = 12
candidate_deciles = 10
min_impurity_decrease = 1e-4

cur.executescript('''
DROP TABLE IF EXISTS dt_nodes;
CREATE TABLE dt_nodes (
    node_id INTEGER PRIMARY KEY,
    depth INTEGER NOT NULL,
    parent_id INTEGER,
    branch_from_parent TEXT,
    is_leaf INTEGER NOT NULL,
    split_feature TEXT,
    split_threshold REAL,
    predicted_class TEXT,
    n_samples INTEGER,
    gini REAL
);

DROP TABLE IF EXISTS dt_node_membership;
CREATE TABLE dt_node_membership (
    sample_id INTEGER PRIMARY KEY,
    node_id INTEGER NOT NULL
);
''')

cur.execute('INSERT INTO dt_node_membership(sample_id, node_id) SELECT sample_id


def compute_node_stats(node_id: int):
    counts = pd.read_sql_query('''
        SELECT d.score_class, COUNT(*) AS n
        FROM dt_node_membership m
        JOIN dt_dataset d ON d.sample_id = m.sample_id
        WHERE m.node_id = ?
        GROUP BY d.score_class
        ORDER BY n DESC, d.score_class ASC;
    ''', conn, params=(int(node_id),))

    n_samples = int(counts['n'].sum()) if not counts.empty else 0
    if n_samples == 0:
        return {'n_samples': 0, 'majority_class': 'middle', 'gini': 0.0}

    probs = counts['n'] / n_samples
    gini = float(1.0 - (probs * probs).sum())
    majority_class = str(counts.iloc[0]['score_class'])
    return {'n_samples': n_samples, 'majority_class': majority_class, 'gini': gi
```

```
root_stats = compute_node_stats(1)
cur.execute('''
    INSERT INTO dt_nodes(node_id, depth, parent_id, branch_from_parent, is_leaf,
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
''', (1, 0, None, None, 1, None, None, root_stats['majority_class'], root_stats[

next_node_id = 1
split_logs = []

for depth in range(max_depth):
    frontier = pd.read_sql_query('''
        SELECT node_id
        FROM dt_nodes
        WHERE depth = ? AND is_leaf = 1
        ORDER BY node_id;
    ''', conn, params=(depth,))

    if frontier.empty:
        break

    for node_id in frontier['node_id'].tolist():
        node_id = int(node_id)
        node_info = compute_node_stats(node_id)

        if node_info['n_samples'] < min_samples_split:
            continue
        if node_info['gini'] <= 1e-12:
            continue

        best_split = pd.read_sql_query('''
            WITH
            node_samples AS (
                SELECT sample_id
                FROM dt_node_membership
                WHERE node_id = ?
            ),
            candidate_points AS (
                SELECT DISTINCT feature, feature_value AS threshold
                FROM (
                    SELECT
                        l.feature,
                        l.feature_value,
                        NTILE(?) OVER (PARTITION BY l.feature ORDER BY l.feature
                    FROM dt_long l
                    JOIN node_samples ns ON ns.sample_id = l.sample_id
                ) t
                WHERE bucket_id BETWEEN 2 AND ? - 1
            ),
            assignments AS (
                SELECT
                    c.feature,
                    c.threshold,
                    l.score_class,
                    CASE WHEN l.feature_value <= c.threshold THEN 'L' ELSE 'R' E
                FROM candidate_points c
                JOIN dt_long l ON l.feature = c.feature
                JOIN node_samples ns ON ns.sample_id = l.sample_id
            ),
            side_class_counts AS (
```

```
    SELECT feature, threshold, side, score_class, COUNT(*) AS n
    FROM assignments
    GROUP BY feature, threshold, side, score_class
),
side_totals AS (
    SELECT
        feature,
        threshold,
        SUM(CASE WHEN side = 'L' THEN n ELSE 0 END) AS n_left,
        SUM(CASE WHEN side = 'R' THEN n ELSE 0 END) AS n_right
    FROM side_class_counts
    GROUP BY feature, threshold
),
valid_splits AS (
    SELECT feature, threshold, n_left, n_right
    FROM side_totals
    WHERE n_left >= ? AND n_right >= ?
),
proportions AS (
    SELECT
        scc.feature,
        scc.threshold,
        scc.side,
        scc.score_class,
        scc.n,
        CASE
            WHEN scc.side = 'L' THEN vs.n_left
            ELSE vs.n_right
        END AS side_n,
        1.0 * scc.n / CASE WHEN scc.side = 'L' THEN vs.n_left ELSE v
    FROM side_class_counts scc
    JOIN valid_splits vs
      ON scc.feature = vs.feature
     AND scc.threshold = vs.threshold
),
side_impurity AS (
    SELECT
        feature,
        threshold,
        side,
        MAX(side_n) AS side_n,
        1.0 - SUM(p * p) AS gini
    FROM proportions
    GROUP BY feature, threshold, side
),
node_total AS (
    SELECT COUNT(*) AS total_n FROM node_samples
),
split_scores AS (
    SELECT
        si.feature,
        si.threshold,
        SUM((1.0 * si.side_n / node_total.total_n) * si.gini) AS wei
    FROM side_impurity si
    CROSS JOIN node_total
    GROUP BY si.feature, si.threshold
)
SELECT feature, threshold, weighted_gini
FROM split_scores
ORDER BY weighted_gini ASC, feature ASC, threshold ASC
```

```
            LIMIT 1;
        ''', conn, params=(node_id, candidate_deciles, candidate_deciles, min_sa

        if best_split.empty:
            continue

        split_feature = str(best_split.iloc[0]['feature'])
        split_threshold = float(best_split.iloc[0]['threshold'])
        split_weighted_gini = float(best_split.iloc[0]['weighted_gini'])

        impurity_decrease = node_info['gini'] - split_weighted_gini
        if impurity_decrease < min_impurity_decrease:
            continue

        left_node_id = next_node_id + 1
        right_node_id = next_node_id + 2
        next_node_id += 2

        cur.execute('''
            UPDATE dt_nodes
            SET is_leaf = 0,
                split_feature = ?,
                split_threshold = ?
            WHERE node_id = ?;
        ''', (split_feature, split_threshold, node_id))

        cur.execute('''
            UPDATE dt_node_membership
            SET node_id = (
                SELECT CASE
                    WHEN l.feature_value <= ? THEN ?
                    ELSE ?
                END
                FROM dt_long l
                WHERE l.sample_id = dt_node_membership.sample_id
                  AND l.feature = ?
            )
            WHERE node_id = ?;
        ''', (split_threshold, left_node_id, right_node_id, split_feature, node_

        left_stats = compute_node_stats(left_node_id)
        right_stats = compute_node_stats(right_node_id)

        cur.execute('''
            INSERT INTO dt_nodes(node_id, depth, parent_id, branch_from_parent,
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
        ''', (left_node_id, depth + 1, node_id, 'L', 1, None, None, left_stats['

        cur.execute('''
            INSERT INTO dt_nodes(node_id, depth, parent_id, branch_from_parent,
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
        ''', (right_node_id, depth + 1, node_id, 'R', 1, None, None, right_stats

        split_logs.append({
            'node_id': node_id,
            'depth': depth,
            'feature': split_feature,
            'threshold': split_threshold,
            'parent_gini': node_info['gini'],
            'split_weighted_gini': split_weighted_gini,
```

```
            'impurity_decrease': impurity_decrease,
            'left_node_id': left_node_id,
            'left_samples': left_stats['n_samples'],
            'right_node_id': right_node_id,
            'right_samples': right_stats['n_samples']
        })

    conn.commit()

nodes_df = pd.read_sql_query('''
    SELECT *
    FROM dt_nodes
    ORDER BY depth, node_id;
''', conn)

display(nodes_df)

split_log_df = pd.DataFrame(split_logs)
if split_log_df.empty:
    print('No valid splits were found with the current constraints.')
else:
    display(split_log_df)

print(f'max_depth={max_depth}, min_samples_split={min_samples_split}, min_sample
```

	node_id	depth	parent_id	branch_from_parent	is_leaf	split_fea
0	1	0	NaN	None	0	Healthy_life_expecta
1	2	1	1.0	L	0	GDP_per_ca
2	3	1	1.0	R	0	GDP_per_ca
3	4	2	2.0	L	0	Perceptions_of_corrup
4	5	2	2.0	R	0	Generc
5	6	2	3.0	L	0	Social_sup
6	7	2	3.0	R	0	Freedom_to_make_life_chc
7	8	3	4.0	L	1	N
8	9	3	4.0	R	1	N
9	10	3	5.0	L	1	N
10	11	3	5.0	R	1	N
11	12	3	6.0	L	1	N
12	13	3	6.0	R	1	N
13	14	3	7.0	L	1	N
14	15	3	7.0	R	1	N

	node_id	depth	feature	threshold	parent_gini	split_weight
0	1	0	Healthy_life_expectancy	0.469000	0.666666	0.
1	2	1	GDP_per_capita	0.648457	0.239527	0.
2	3	1	GDP_per_capita	1.229000	0.607713	0.
3	4	2	Perceptions_of_corruption	0.179550	0.145161	0.
4	5	2	Generosity	0.136560	0.480151	0.
5	6	2	Social_support	0.777110	0.599114	0.
6	7	2	Freedom_to_make_life_choices	0.498465	0.267716	0.

```
max_depth=3, min_samples_split=30, min_samples_leaf=12, min_impurity_decrease=0.0
001
```

In [8]:
```python
# 3) Evaluate multi-level tree predictions in SQL.
cur.executescript('''
DROP VIEW IF EXISTS dt_predictions;
CREATE VIEW dt_predictions AS
SELECT
    m.sample_id,
    d.score_class AS y_true,
    n.predicted_class AS y_pred,
    n.node_id AS leaf_node_id,
    n.depth AS leaf_depth
FROM dt_node_membership m
JOIN dt_dataset d ON d.sample_id = m.sample_id
JOIN dt_nodes n ON n.node_id = m.node_id
WHERE n.is_leaf = 1;
''')
conn.commit()

tree_eval = pd.read_sql_query('''
SELECT
    AVG(CASE WHEN y_true = y_pred THEN 1.0 ELSE 0.0 END) AS accuracy,
    AVG(CASE WHEN y_true = y_pred THEN 0.0 ELSE 1.0 END) AS zero_one_loss,
    COUNT(*) AS n_samples
FROM dt_predictions;
''', conn)
display(tree_eval)

leaf_summary = pd.read_sql_query('''
SELECT
    n.node_id,
    n.depth,
    n.parent_id,
    n.branch_from_parent,
    n.predicted_class,
    n.n_samples,
    n.gini
FROM dt_nodes n
WHERE n.is_leaf = 1
ORDER BY n.depth, n.node_id;
''', conn)
display(leaf_summary)
```

```
confusion_df = pd.read_sql_query('''
SELECT
    y_true,
    y_pred,
    COUNT(*) AS n
FROM dt_predictions
GROUP BY y_true, y_pred
ORDER BY y_true, y_pred;
''', conn)
display(confusion_df)
```

	accuracy	zero_one_loss	n_samples
0	0.733675	0.266325	781

	node_id	depth	parent_id	branch_from_parent	predicted_class	n_samples	
0	8	3	4	L	low	149	0.101
1	9	3	4	R	low	16	0.429
2	10	3	5	L	low	24	0.000
3	11	3	5	R	middle	22	0.512
4	12	3	6	L	low	50	0.485
5	13	3	6	R	middle	319	0.571
6	14	3	7	L	high	84	0.444
7	15	3	7	R	high	117	0.066

	y_true	y_pred	n
0	high	high	169
1	high	low	1
2	high	middle	91
3	low	low	207
4	low	middle	53
5	middle	high	32
6	middle	low	31
7	middle	middle	197

C) K-means Clustering (k=3, SQL distance + SQL centroid updates)

Features (as required):

- `Score`
- `GDP_per_capita`
- `Social_support`

- `Healthy_life_expectancy`
- `Freedom_to_make_life_choices`
- `Generosity`
- `Perceptions_of_corruption`

Workflow:

1. Min-max normalize all 7 features in SQL
2. Initialize 3 centroids from three score buckets
3. Iterate assignment/update steps (all core math in SQL)
4. Report cluster size, centroid values, and SSE

In [9]:
```
# 1) Prepare normalized points table.
cur.executescript('''
DROP TABLE IF EXISTS kmeans_points;
CREATE TABLE kmeans_points AS
WITH stats AS (
    SELECT
        MIN(Score) AS min_score,
        MAX(Score) AS max_score,
        MIN(GDP_per_capita) AS min_gdp,
        MAX(GDP_per_capita) AS max_gdp,
        MIN(Social_support) AS min_social,
        MAX(Social_support) AS max_social,
        MIN(Healthy_life_expectancy) AS min_health,
        MAX(Healthy_life_expectancy) AS max_health,
        MIN(Freedom_to_make_life_choices) AS min_freedom,
        MAX(Freedom_to_make_life_choices) AS max_freedom,
        MIN(Generosity) AS min_generosity,
        MAX(Generosity) AS max_generosity,
        MIN(Perceptions_of_corruption) AS min_corr,
        MAX(Perceptions_of_corruption) AS max_corr
    FROM happyness
)
SELECT
    rowid AS point_id,
    Country,
    Year,
    (Score - min_score) / NULLIF(max_score - min_score, 0) AS f1_score,
    (GDP_per_capita - min_gdp) / NULLIF(max_gdp - min_gdp, 0) AS f2_gdp,
    (Social_support - min_social) / NULLIF(max_social - min_social, 0) AS f3_soc
    (Healthy_life_expectancy - min_health) / NULLIF(max_health - min_health, 0)
    (Freedom_to_make_life_choices - min_freedom) / NULLIF(max_freedom - min_free
    (Generosity - min_generosity) / NULLIF(max_generosity - min_generosity, 0) A
    (Perceptions_of_corruption - min_corr) / NULLIF(max_corr - min_corr, 0) AS f
FROM happyness, stats;

DROP TABLE IF EXISTS kmeans_centroids;
CREATE TABLE kmeans_centroids (
    cluster_id INTEGER PRIMARY KEY,
    f1_score REAL,
    f2_gdp REAL,
    f3_social REAL,
    f4_health REAL,
    f5_freedom REAL,
    f6_generosity REAL,
    f7_corruption REAL
```

```
);

INSERT INTO kmeans_centroids(cluster_id, f1_score, f2_gdp, f3_social, f4_health,
WITH seeded AS (
    SELECT
        point_id,
        f1_score, f2_gdp, f3_social, f4_health, f5_freedom, f6_generosity, f7_co
        NTILE(3) OVER (ORDER BY f1_score) AS seed_cluster
    FROM kmeans_points
)
SELECT
    seed_cluster AS cluster_id,
    AVG(f1_score),
    AVG(f2_gdp),
    AVG(f3_social),
    AVG(f4_health),
    AVG(f5_freedom),
    AVG(f6_generosity),
    AVG(f7_corruption)
FROM seeded
GROUP BY seed_cluster
ORDER BY seed_cluster;
''')
conn.commit()

display(pd.read_sql_query('SELECT * FROM kmeans_centroids ORDER BY cluster_id;',
```

	cluster_id	f1_score	f2_gdp	f3_social	f4_health	f5_freedom	f6_generosity	f7
0	1	0.280472	0.283214	0.501152	0.334491	0.454907	0.257932	
1	2	0.527525	0.507107	0.676135	0.570788	0.538167	0.230512	
2	3	0.779383	0.677041	0.791886	0.705512	0.711450	0.294137	

In [10]:
```
# 2) Iterate k-means until convergence or max iterations.
max_iter = 50
tol = 1e-8
history = []

for i in range(max_iter):
    # Assignment step: choose nearest centroid for each point.
    cur.executescript('''
    DROP TABLE IF EXISTS kmeans_assignments;
    CREATE TABLE kmeans_assignments AS
    WITH distances AS (
        SELECT
            p.point_id,
            c.cluster_id,
            (
                POWER(p.f1_score - c.f1_score, 2) +
                POWER(p.f2_gdp - c.f2_gdp, 2) +
                POWER(p.f3_social - c.f3_social, 2) +
                POWER(p.f4_health - c.f4_health, 2) +
                POWER(p.f5_freedom - c.f5_freedom, 2) +
                POWER(p.f6_generosity - c.f6_generosity, 2) +
                POWER(p.f7_corruption - c.f7_corruption, 2)
            ) AS sq_distance,
```

```
            ROW_NUMBER() OVER (
                PARTITION BY p.point_id
                ORDER BY
                    (
                        POWER(p.f1_score - c.f1_score, 2) +
                        POWER(p.f2_gdp - c.f2_gdp, 2) +
                        POWER(p.f3_social - c.f3_social, 2) +
                        POWER(p.f4_health - c.f4_health, 2) +
                        POWER(p.f5_freedom - c.f5_freedom, 2) +
                        POWER(p.f6_generosity - c.f6_generosity, 2) +
                        POWER(p.f7_corruption - c.f7_corruption, 2)
                    ) ASC,
                    c.cluster_id ASC
            ) AS rn
        FROM kmeans_points p
        CROSS JOIN kmeans_centroids c
    )
    SELECT point_id, cluster_id, sq_distance
    FROM distances
    WHERE rn = 1;
    ''')

    old_centroids = pd.read_sql_query('SELECT * FROM kmeans_centroids ORDER BY c

    # Update step: recompute centroids from current assignments.
    cur.executescript('''
    DROP TABLE IF EXISTS kmeans_new_centroids;
    CREATE TABLE kmeans_new_centroids AS
    SELECT
        a.cluster_id,
        AVG(p.f1_score) AS f1_score,
        AVG(p.f2_gdp) AS f2_gdp,
        AVG(p.f3_social) AS f3_social,
        AVG(p.f4_health) AS f4_health,
        AVG(p.f5_freedom) AS f5_freedom,
        AVG(p.f6_generosity) AS f6_generosity,
        AVG(p.f7_corruption) AS f7_corruption
    FROM kmeans_assignments a
    JOIN kmeans_points p ON p.point_id = a.point_id
    GROUP BY a.cluster_id;

    DROP TABLE IF EXISTS kmeans_centroids_next;
    CREATE TABLE kmeans_centroids_next AS
    SELECT
        c.cluster_id,
        COALESCE(n.f1_score, c.f1_score) AS f1_score,
        COALESCE(n.f2_gdp, c.f2_gdp) AS f2_gdp,
        COALESCE(n.f3_social, c.f3_social) AS f3_social,
        COALESCE(n.f4_health, c.f4_health) AS f4_health,
        COALESCE(n.f5_freedom, c.f5_freedom) AS f5_freedom,
        COALESCE(n.f6_generosity, c.f6_generosity) AS f6_generosity,
        COALESCE(n.f7_corruption, c.f7_corruption) AS f7_corruption
    FROM kmeans_centroids c
    LEFT JOIN kmeans_new_centroids n
      ON c.cluster_id = n.cluster_id;

    DELETE FROM kmeans_centroids;
    INSERT INTO kmeans_centroids
    SELECT * FROM kmeans_centroids_next;
    ''')
```

```
    new_centroids = pd.read_sql_query('SELECT * FROM kmeans_centroids ORDER BY c

    merged = old_centroids.merge(new_centroids, on='cluster_id', suffixes=('_old
    movement = 0.0
    for f in ['f1_score', 'f2_gdp', 'f3_social', 'f4_health', 'f5_freedom', 'f6_
        movement += ((merged[f'{f}_old'] - merged[f'{f}_new']) ** 2).sum()

    sse = pd.read_sql_query('SELECT SUM(sq_distance) AS sse FROM kmeans_assignme
    history.append({'iter': i, 'centroid_movement': float(movement), 'sse': floa

    conn.commit()

    if movement < tol:
        break

kmeans_history_df = pd.DataFrame(history)
display(kmeans_history_df.tail(10))
```

	iter	centroid_movement	sse
2	2	0.001476	105.306612
3	3	0.001161	104.706760
4	4	0.000454	104.317727
5	5	0.000055	104.218194
6	6	0.000044	104.201792
7	7	0.000008	104.189082
8	8	0.000008	104.186599
9	9	0.000011	104.183307
10	10	0.000007	104.180035
11	11	0.000000	104.178796

In [11]:
```
# 3) Final clustering outputs: centroid table, cluster sizes, and sample members
final_centroids_df = pd.read_sql_query('SELECT * FROM kmeans_centroids ORDER BY
display(final_centroids_df)

cluster_size_df = pd.read_sql_query('''
SELECT cluster_id, COUNT(*) AS n_points
FROM kmeans_assignments
GROUP BY cluster_id
ORDER BY cluster_id;
''', conn)
display(cluster_size_df)

cluster_samples_df = pd.read_sql_query('''
SELECT
    a.cluster_id,
    p.Country,
    p.Year,
    ROUND(a.sq_distance, 6) AS sq_distance
FROM kmeans_assignments a
JOIN kmeans_points p ON p.point_id = a.point_id
ORDER BY a.cluster_id, a.sq_distance ASC
```

```
LIMIT 30;
''', conn)
display(cluster_samples_df)

final_sse_df = pd.read_sql_query('SELECT SUM(sq_distance) AS total_sse FROM kmea
display(final_sse_df)
```

	cluster_id	f1_score	f2_gdp	f3_social	f4_health	f5_freedom	f6_generosity	f7
0	1	0.293135	0.242214	0.467427	0.285865	0.459903	0.279805	
1	2	0.568943	0.553138	0.717788	0.617485	0.555904	0.210711	
2	3	0.831655	0.743099	0.815880	0.751731	0.792921	0.368550	

	cluster_id	n_points
0	1	247
1	2	394
2	3	140

	cluster_id	Country	Year	sq_distance
0	1	Senegal	2016	0.008529
1	1	Guinea	2019	0.012925
2	1	Guinea	2018	0.027332
3	1	Burkina Faso	2016	0.030192
4	1	Tanzania	2016	0.033146
5	1	Senegal	2015	0.033692
6	1	Sierra Leone	2019	0.035087
7	1	Mauritania	2015	0.035799
8	1	Burkina Faso	2015	0.038016
9	1	Mali	2016	0.038756
10	1	Togo	2019	0.039325
11	1	Ethiopia	2017	0.039634
12	1	Niger	2019	0.041224
13	1	Burkina Faso	2017	0.041910
14	1	Zimbabwe	2016	0.043042
15	1	Yemen	2015	0.043951
16	1	Gambia	2019	0.045160
17	1	Ivory Coast	2019	0.045393
18	1	Guinea	2017	0.045425
19	1	Ghana	2015	0.048837
20	1	Niger	2017	0.050441
21	1	Congo (Brazzaville)	2015	0.050491
22	1	Ethiopia	2018	0.053579
23	1	Niger	2018	0.053585
24	1	Zambia	2016	0.055655
25	1	Burkina Faso	2018	0.058439
26	1	Uganda	2019	0.058572
27	1	Zimbabwe	2017	0.058739
28	1	Mali	2015	0.058763
29	1	Ghana	2017	0.059143

	total_sse
0	104.178796